

File I/O

COMPLETE LEARNING GUIDE

What You'll Learn

Reading and writing files is a practical, everyday skill tested in both coding rounds (parsing input files) and system-design discussions (log files, config loading), and Java offers two overlapping APIs for it: the original `java.io` streams-and-readers model, and the more modern, more convenient `java.nio.file` package introduced in Java 7. This guide covers both — `FileReader/BufferedReader` for text input, `FileWriter/BufferedWriter` for text output, the try-with-resources pattern that makes file handling exception-safe, the modern `Path/Files` utility class that replaces most manual stream plumbing with one-line calls, basic object serialization via the `Serializable` interface, and the `IOException` hierarchy you must handle correctly. Interviewers commonly ask candidates to read a file line by line and process it — knowing the idiomatic, resource-safe way to do this is a baseline expectation.

1. Reading Text Files: `FileReader` and `BufferedReader`

`FileReader` reads a file character by character, which is technically correct but extremely slow for anything beyond a trivial file because each read may involve a system call; `BufferedReader` wraps any `Reader` and adds an internal buffer, batching reads into larger chunks and adding the crucial `readLine()` method that `FileReader` alone doesn't provide. The idiomatic pattern wraps a `FileReader` in a `BufferedReader` and reads line by line in a while loop, checking for null (`readLine()` returns null at end-of-file, not an exception) as the loop's termination condition.

JAVA

```
try (BufferedReader reader = new BufferedReader(new FileReader("input.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.err.println("Failed to read file: " + e.getMessage());
}
```

2. Writing Text Files: `FileWriter` and `BufferedWriter`

Writing mirrors reading: `FileWriter` writes characters directly to a file (recreating it by default, or appending if constructed with the `append=true` flag), and `BufferedWriter` wraps it to batch writes for performance. Unlike `println()`, `BufferedWriter`'s `newLine()` method writes the platform-specific line separator (`\r\n` on Windows, `\n` on Unix) rather than hardcoding one, which matters for files meant to be read correctly across operating systems.

JAVA

```
try (BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt",
true))) { // true = append mode, don't overwrite existing content
    writer.write("New log entry");
    writer.newLine(); // platform-correct line separator
} catch (IOException e) {
    System.err.println("Failed to write file: " + e.getMessage());
}
```

3. try-with-resources Is Mandatory, Not Optional, for File I/O

Every file-handling example above uses try-with-resources, and this is not stylistic preference — file handles are a genuinely limited operating-system resource, and a program that opens files without reliably closing them will eventually exhaust the OS's file descriptor limit and start failing to open new files, a bug that's notoriously hard to diagnose because it only manifests after many operations have already succeeded. try-with-resources guarantees close() is called even when readLine() or write() throws partway through, which a naive try/catch without finally-based closing does not.

4. The Modern Approach: java.nio.file.Path and Files

Since Java 7, the java.nio.file package offers a higher-level, more convenient API: Path represents a file-system location (replacing the older, less capable java.io.File), and the Files utility class provides static methods for nearly every common file operation in a single call — reading an entire small file into a String or a List<String> of lines, writing a String to a file, checking existence, copying, deleting, and walking directory trees — without manually managing a Reader/Writer at all for simple cases.

JAVA

```
Path path = Path.of("input.txt");

List<String> lines = Files.readAllLines(path); // whole file, one call
String content = Files.readString(path); // Java 11+, whole file
as a String

Files.writeString(Path.of("output.txt"), "Hello, file!"); // Java 11+, one call
to write

boolean exists = Files.exists(path);
Files.copy(path, Path.of("backup.txt"));
```

5. Streaming Large Files Without Loading Everything Into Memory

Files.readAllLines() and Files.readString() load the entire file into memory at once, which is fine for small config or input files but dangerous for large files (multi-gigabyte logs) where it can exhaust heap memory. For large files, Files.lines(path) returns a lazily-evaluated Stream<String> that reads and processes one line at a time without materializing the whole file in memory — but critically, that stream must be closed (via try-with-resources) because, unlike most Stream sources, it holds an open file handle underneath.

JAVA

```
try (Stream<String> lines = Files.lines(Path.of("huge-log.txt"))) {
    long errorCount = lines.filter(line -> line.contains("ERROR")).count();
    System.out.println("Errors: " + errorCount);
} catch (IOException e) {
    System.err.println("Failed to process file: " + e.getMessage());
}
```

6. Basic Object Serialization

A class that implements the marker interface `Serializable` (no methods to implement — it simply signals intent to the JVM) can have its entire object graph written to a byte stream via `ObjectOutputStream.writeObject()` and reconstructed later via `ObjectInputStream.readObject()`, commonly used for simple persistence or caching. Every non-transient field must itself be `Serializable` (or primitive) or serialization throws at runtime; marking a field transient excludes it deliberately (common for passwords, caches, or non-serializable fields like a `Thread` or a database `Connection`). A `serialVersionUID` field should be declared explicitly on every `Serializable` class — without it, the JVM auto-generates one from the class's structure, and any later change to the class can silently break deserialization of previously-saved objects with an `InvalidClassException`.

7. The `IOException` Hierarchy and Handling It Correctly

`IOException` is a checked exception, and its common subclasses carry more specific meaning worth catching separately when the handling differs: `FileNotFoundException` (the file doesn't exist, or can't be opened for the requested access), `EOFException` (unexpected end of stream during a structured read, distinct from `readLine()`'s null-return convention), and `UncheckedIOException` (a wrapper used inside Stream pipelines, like `Files.lines()`, where checked exceptions can't cross a functional interface's throws-free method signature). Since `FileNotFoundException` extends `IOException`, it must be caught before a broader catch (`IOException e`) block, or the compiler flags the more specific catch as unreachable.

Common Pitfalls

- Opening a `FileReader/FileWriter` without `try-with-resources`, leaking file handles that eventually exhaust the OS's file descriptor limit.
- Using `Files.readAllLines()` or `readString()` on a very large file, loading the entire contents into memory and risking an `OutOfMemoryError` — use `Files.lines()` (streamed) instead.
- Forgetting `Files.lines()` returns a `Stream` that itself holds an open file handle and must be closed with `try-with-resources`, unlike most other `Stream` sources.
- Checking `readLine()`'s return for an exception instead of null — end-of-file is signaled by a null return, not an exception.
- Catching `IOException` before the more specific `FileNotFoundException` in the same try block, which the compiler rejects as unreachable code.

- Marking a field holding a non-serializable resource (a Thread, a database Connection, a Socket) as part of a Serializable class without declaring it transient, causing serialization to throw at runtime.
- Omitting an explicit serialVersionUID, letting the JVM auto-generate one that changes whenever the class's structure changes, silently breaking deserialization of old saved data.

Key Takeaways

- Always wrap file streams in try-with-resources — file handles are a limited OS resource and must be reliably closed even when an exception occurs.
- BufferedReader/BufferedWriter add line-based methods and batch I/O for performance over raw FileReader/FileWriter.
- The modern java.nio.file.Files utility class replaces most manual stream plumbing with single-call methods (readAllLines, readString, writeString, copy).
- Use Files.lines() (streamed, lazy) instead of Files.readAllLines() (loads everything) for large files, and remember it needs to be closed too.
- Serializable is a marker interface; every non-transient field must also be Serializable, and an explicit serialVersionUID guards against silent deserialization breakage.
- readLine() signals end-of-file with a null return, not an exception — check for null, don't wrap it in a try/catch expecting an error.

Learn More

Oracle Java Tutorials — Basic I/O

<https://docs.oracle.com/javase/tutorial/essential/io/index.html>

Oracle Java SE Docs — java.nio.file.Files

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/nio/file/Files.html>

Baeldung — Reading a File in Java

<https://www.baeldung.com/reading-file-in-java>